

Reviewer Pack — Minimal Ehrhart Rank and SAT Clause Depth Correlation

Entry 0e11d80338a6 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-07 17:22:35 UTC

Minimal Ehrhart Rank and SAT Clause Depth Correlation

Entry ID: 0e11d80338a6 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-07 17:22:35 UTC

1. Conjecture

Field A (mathematical branch): Polyhedral Geometry (Ehrhart Theory) **Field B** (complexity object): Boolean Satisfiability (Clause Complexity)

Statement:

For every d -regular graph G , the minimal Ehrhart rank ($m_Ehr(G)$) of its associated binary polytope P_G is linearly correlated with its SAT clause depth $c(G)$, such that $m_Ehr(G) = \Theta(c(G))$.

Rationale (proposer's reasoning):

Ehrhart theory studies the integer points in convex polyhedra and their relationship to combinatorial problems. The conjecture bridges this area with complexity theory by hypothesizing a correlation between the Ehrhart rank of a graph's associated binary polytope and its clause depth, which could reveal new structural insights into SAT solving.

Taxonomy category: Polyhedral_Ehrhart_Boolean_SAT (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 20ffcbadb6ae4d8d

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For a d -regular graph G with $n \leq 40$, if the computed Ehrhart rank $m_Ehr(G)$ of its binary polytope P_G and clause depth $c(G)$ are linearly correlated with a correlation coefficient $r \geq 0.99$ within $\pm 1\%$ error margin over 30 seeds.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
ALGEBRIZATION	SAFE	0.90	SAFE	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 8 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - 'Ehrhart Theory' AND 'SAT clause depth' AND correlation' - 'polyhedral geometry' AND 'binary polytope' AND 'SAT clause complexity' - 'minimal Ehrhart rank' AND 'd-regular graph' AND 'Boolean satisfiability'

Top relevant hits considered: - [http://arxiv.org/abs/1004.0653v2] Exact Ramsey Theory: Green-Tao numbers and SAT - [http://arxiv.org/abs/2211.17129v2] Ehrhart Limits - [http://arxiv.org/abs/2605.09347v1] Dsat: A Native SAT Solver for Discrete Logic - [http://arxiv.org/abs/1604.01175v4] On the Combinatorial Complexity of Approximating Polytopes - [http://arxiv.org/abs/2303.09586v2] Optimal Volume-Sensitive Bounds for Polytope Approximation - [http://arxiv.org/abs/math/0106093v2] On the Complexity of Polytope Isomorphism Problems - [http://arxiv.org/abs/0904.3035v2] Additive number theory and inequalities in Ehrhart theory - [http://arxiv.org/abs/2005.03259v2] On the Gorenstein property of the Ehrhart ring of the stable set polytope of an h-perfect graph

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=124, elapsed=240.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_d_regular_graph(n, d):
        if (d * n) % 2 != 0 or d >= n:
            return None
        adj_matrix = [[0] * n for _ in range(n)]
        edges_added = 0
        while edges_added < d * n // 2:
            u = random.randint(0, n-1)
            v = random.randint(0, n-1)
            if u != v and adj_matrix[u][v] == 0:
                adj_matrix[u][v] = 1
                adj_matrix[v][u] = 1
                edges_added += 1
        return adj_matrix

    def zonotope_construction(adj_matrix):
        n = len(adj_matrix)
        vertices = []
        for i in range(n):
            vertex = [0] * n
            vertex[i] = 1
            vertices.append(vertex)
```

```

        vertex[i] = -1
        vertices.append(vertex)
    return vertices

def ehrhart_rank(vertices):
    n = len(vertices[0])
    count = 0
    for i in range(2**n):
        point = [0] * n
        for j in range(n):
            if (i >> j) & 1:
                point[j] = 1
        if all(all(point[k] + vertices[m][k] >= 0 for k in range(n)) for m in range(len(vertices))):
            count += 1
    return count

def sat_clause_depth(adj_matrix):
    n = len(adj_matrix)
    clauses = []
    for i in range(n):
        clause = [i]
        for j in range(i+1, n):
            if adj_matrix[i][j] == 1:
                clause.append(j)
        clauses.append(clause)
    return len(clauses)

def correlation_coefficient(x, y):
    mean_x = sum(x) / len(x)
    mean_y = sum(y) / len(y)
    cov_xy = sum((x_i - mean_x) * (y_i - mean_y) for x_i, y_i in zip(x, y)) / len(x)
    std_x = math.sqrt(sum((x_i - mean_x)**2 for x_i in x) / len(x))
    std_y = math.sqrt(sum((y_i - mean_y)**2 for y_i in y) / len(y))
    return cov_xy / (std_x * std_y)

n_values = [5, 10, 15, 20, 30, 40]
m_Ehr_values = []
c_G_values = []

for n in n_values:
    G = generate_d_regular_graph(n, 2)
    if G is None:
        continue
    P_G = zonotope_construction(G)
    m_Ehr = ehrhart_rank(P_G)
    c_G = sat_clause_depth(G)
    m_Ehr_values.append(m_Ehr)
    c_G_values.append(c_G)

if len(m_Ehr_values) < 30:
    return {
        "metric_name": "correlation_coefficient",
        "metric_value": None,
        "instances_tested": len(m_Ehr_values),
        "n_max": max(n_values),
        "conjecture_holds": False,
    }

```

```

        "counterexample": "not_enough_instances"
    }

    r = correlation_coefficient(m_Ehr_values, c_G_values)
    return {
        "metric_name": "correlation_coefficient",
        "metric_value": r,
        "instances_tested": len(m_Ehr_values),
        "n_max": max(n_values),
        "conjecture_holds": abs(r) >= 0.99,
        "counterexample": ""
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43]
    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_r = sum(r["metric_value"] for r in results if r["metric_value"] is not None) / len(results)
    std_r = math.sqrt(sum((r["metric_value"] - mean_r)**2 for r in results if r["metric_value"] is not None) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_r} std={std_r} support_fraction={support_fraction}")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_r} std={std_r} support_fraction={support_fraction}")
    else:
        first_failing_seed = next((r["seed"] for r in results if not r["conjecture_holds"]), None)
        print(f"RESULT: FALSIFIED counterexample='correlation_coefficient' first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test timed out before producing data, which means that the pre-registered support condition could not be unambiguously met. | next: NONE

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	13117
2	preregistration	ollama_remote	glm4:latest	0	0	14021
3	novelty	ollama_remote	glm4:latest	0	0	8428
4	novelty	ollama_remote	glm4:latest	0	0	19921
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16976
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12055
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9260
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14380
9	judge	ollama_remote	glm4:latest	0	0	16724

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 124882 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/0e11d80338a6.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/0e11d80338a6.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/0e11d80338a6.tar.gz` (if generated)